

Potruga za blagom

Ograničenje vremena: 8 s Ograničenje memorije: 256 MiB

Tražite davno izgubljeno blago na golemom polju veličine $N \times N$ pomoću čarobnog kompasa. Na polju su skrivena ukupno $K \leq 3$ sanduka s blagom, pri čemu se svaki nalazi u različitom polju. Vaš je cilj jednostavan: pronaći ih sve!

Kada postavite kompasa na neko polje, on pronalazi najkraći put do sanduka s blagom koristeći samo pomake ulijevo, gore, udesno i dolje (odnosno do najbližeg sanduka prema Manhattanovoj udaljenosti). Zatim pokazuje smjer prvog koraka na tom putu. Ako postoji više mogućih prvih koraka koji vode do najbližeg sanduka (ili više njih), kompas će vratiti sve takve smjerove. Ako se na tom polju nalazi blago, kompas će to naznačiti.

Kad pronađete sanduk s blagom, ispraznite njegov sadržaj, ali ga ne možete ukloniti jer je pretežak. Kompas ne razlikuje puni i prazni sanduk. Uvijek pokazuje prema najbližem sanduku, bez obzira na to je li on pun ili prazan.

Pokušajte pronaći položaj svih sanduka koristeći što manje upita kompasu.

Zadatak

Ovo je interaktivan zadatak. U svakom testnom primjeru (tj. pri svakom pokretanju vašeg programa) vaš program mora riješiti više potraga za blagom. S ocjenjivačem komunicirate pomoću biblioteke koju su pripremili organizatori. Ta biblioteka sadrži sljedeće deklaracije:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — pozovite ovu funkciju za početak sljedeće potrage za blagom. Ona će postaviti veličinu mreže u varijablu *N* te broj blaga u *K*. Ako više nema potraga koje treba riješiti tijekom trenutnog pokretanja programa, funkcija će postaviti *N* i *K* na -1 ; u tom slučaju trebate završiti program s izlaznim kodom 0. Napomena: ovu funkciju možete pozvati i prije nego što pronađete sva blaga u trenutnoj potrazi, primjerice ako želite ostvariti samo djelomičan broj bodova.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — to su konstante koje se koriste u povratnim vrijednostima funkcije *Query* (vidi dolje).
- **int** *Query*(**int** *x*, **int** *y*) — ako se u polju s koordinatama (*x*, *y*) nalazi blago, funkcija vraća *TREASURE*. U suprotnom vraća zbroj jedne ili više konstanti *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* i *DIR_DOWN*, označavajući koje smjerove kompasa vraća kada se postavi na polje (*x*, *y*). Koordinate *x* i *y* moraju biti cijeli brojevi iz raspona od 0 do $N - 1$. (Napomena: u ovom zadatku koordinata *y* raste od vrha prema dnu.)

Nakon što *NextHunt* vrati $N = K = -1$, vaš program više ne smije pozivati ni *NextHunt* ni *Query*. Također, *Query* se ne smije pozivati prije prvog poziva funkcije *NextHunt*. Ako vaš program prekrši ta pravila, napravi više od 1000 upita tijekom jedne potrage ili proslijedi vrijednosti *x* i/ili *y* izvan dopuštenog raspona pri pozivu funkcije *Query*, biblioteka će prekinuti izvršavanje programa te će za taj testni primjer biti dodijeljena presuda **RTE** (Run-Time Error).

Smatra se da je blago pronađeno ako je vaš program barem jednom postavio upit za polje koje ga sadrži. Ako vaš program pozove *NextHunt* prije nego što pronađe sva blaga u trenutnoj potrazi, to se ne smatra pogreškom, ali će utjecati na ostvareni broj bodova (više o tome u odjeljku o bodovanju).

Za korištenje biblioteke vaš program treba uključiti zaglavlje `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

To zaglavlje možete preuzeti ovdje: `treasurehuntlib.h`.

Kako biste lakše razvili svoje rješenje, dostupna je i jednostavna implementacija biblioteke: `treasurehuntlib-public.cpp`. Da biste je preveli zajedno sa svojim programom, jednostavno dodajte njezino ime kao parametar prevoditelju, na primjer:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

ako je `foo.cpp` naziv datoteke koja sadrži vaše rješenje.

Implementacija u datoteci `treasurehuntlib-public.cpp` uz gore navedene deklaracije podržava i funkciju `void InitFromFile(const char *fileName)`, koja učitava popis potraga iz datoteke i omogućuje vam igranje na njima umjesto na nasumično generiranim potragama. Više pojedinosti možete pronaći unutar same datoteke `treasurehuntlib-public.cpp`.

Na sustavu za ocjenjivanje koristit će se druga implementacija biblioteke, stoga ne smijete pretpostavljati ništa o načinu na koji je implementirana. Ipak, možete pretpostaviti da ne dodaje u globalni prostor imena nikakve deklaracije osim gore navedenih (*NextHunt*, *Query* i pet konstanti).

Vaš program ne smije čitati sa standardnog ulaza niti pisati na standardni izlaz, jer će se oni na sustavu za ocjenjivanje koristiti za komunikaciju između implementacije biblioteke i ostatka okruženja za ocjenjivanje.

Ulaz

Ograničenja

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Tijekom jednog pokretanja programa bit će najviše 100 000 potraga za blagom.
- Tijekom jedne potrage smijete postaviti najviše 1000 upita.
- Sustav za ocjenjivanje nije adaptivan.

Podzadaci

- Podzadatak 1 (10 bodova): $K = 1$.
- Podzadatak 2 (30 bodova): $K = 2$.
- Podzadatak 3 (60 bodova): $K = 3$.

Bodovanje

Jedan podzadatak može sadržavati više testnih primjera (više pokretanja vašeg programa), a svaki testni primjer može sadržavati više potraga za blagom. Za potrebe bodovanja sve se potrage unutar istog podzadatka promatraju zajedno, bez obzira na to kako su raspoređene po testnim primjerima.

Za i -tu potragu neka je veličina mreže $N_i \times N_i$, broj blaga K_i , broj upita koje je vaš program postavio Q_i , a broj pronađenih blaga F_i . Nadalje, neka je S ukupan broj bodova za taj podzadatak. Tada je broj bodova koji vaš program dobiva za taj podzadatak:

- Ako je vaš program uvijek pronašao sva blaga (tj. ako za svaki i vrijedi $F_i = K_i$), broj bodova ovisi o vrijednosti $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ bodova, gdje je } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t_i \geq 20. \end{cases}$$

- Ako vaš program nije uvijek pronašao sva blaga (tj. postoji i za koji vrijedi $F_i < K_i$), dobiva

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ bodova.}$$

Drugim riječima, polovicu bodova dobivate za pronalazak svih blaga, a drugu polovicu za to da ih pronađete koristeći što manje upita. Za maksimalan broj bodova vaše rješenje mora pronaći sva blaga u najviše $11 \lceil \log_2 N_i \rceil$ upita. Između $11 \lceil \log_2 N_i \rceil$ i $20 \lceil \log_2 N_i \rceil$ upita broj bodova linearno se smanjuje, a ako vaše rješenje napravi više od $20 \lceil \log_2 N_i \rceil$ upita, dobit će samo prvu polovicu bodova za pronalazak svih blaga. (Simboli $\lceil \cdot \rceil$ označavaju zaokruživanje vrijednosti $\log_2 N_i$ na najbliži veći cijeli broj.)

Ako navedene formule daju necijeli broj bodova za podzadatak, rezultat se zaokružuje na najbliži cijeli broj.

Ako vaš program završi s greškom tijekom izvođenja ili se ne pridržava prethodno opisanog protokola korištenja biblioteke, dobit će 0 bodova za cijeli podzadatak. Kako biste ostvarili djelomične bodove za pronalazak samo dijela blaga, vaš program mora uredno završiti trenutnu potragu pozivom funkcije *NextHunt*.

Primjer

Poziv	Povratna vrijednost
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$